DO NOW — RECALL

# Lesson 118 · Extreme Programming and RAD

Name: _____ Class: _____ Date: _____

**1. Which lifecycle uses relatively fixed sequential stages?**

**2. For the binary value 1010, what is its decimal value?**

**3. A linked-list node has data and what other essential field?**

**4. In a BST, where is a value less than the current node placed?**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

### 1. Model answer

Waterfall.

### 2. Model answer

10.

### 3. Model answer

A link/reference to the next node (for a singly linked list).

### 4. Model answer

In its left subtree (under the chosen no-duplicate convention).

### By the end, you can…

• Test-driven development, pair programming, refactoring and continuous integration.

• Rapid application development uses prototyping and stakeholder collaboration.

• Consider user availability, architecture, maintainability and regulatory oversight.

### Vocabulary — know these terms

XP — Agile method emphasising disciplined engineering practices.

TDD — Write a failing test then implement/refactor.

RAD — Development centred on rapid prototyping and user feedback.

refactoring — Improving structure without changing observable behaviour.

NEW LEARNING

# Read, annotate and explain

### XP practices

Test-driven development, pair programming, refactoring and continuous integration.

### RAD approach

Rapid application development uses prototyping and stakeholder collaboration.

### Balance cost and fit

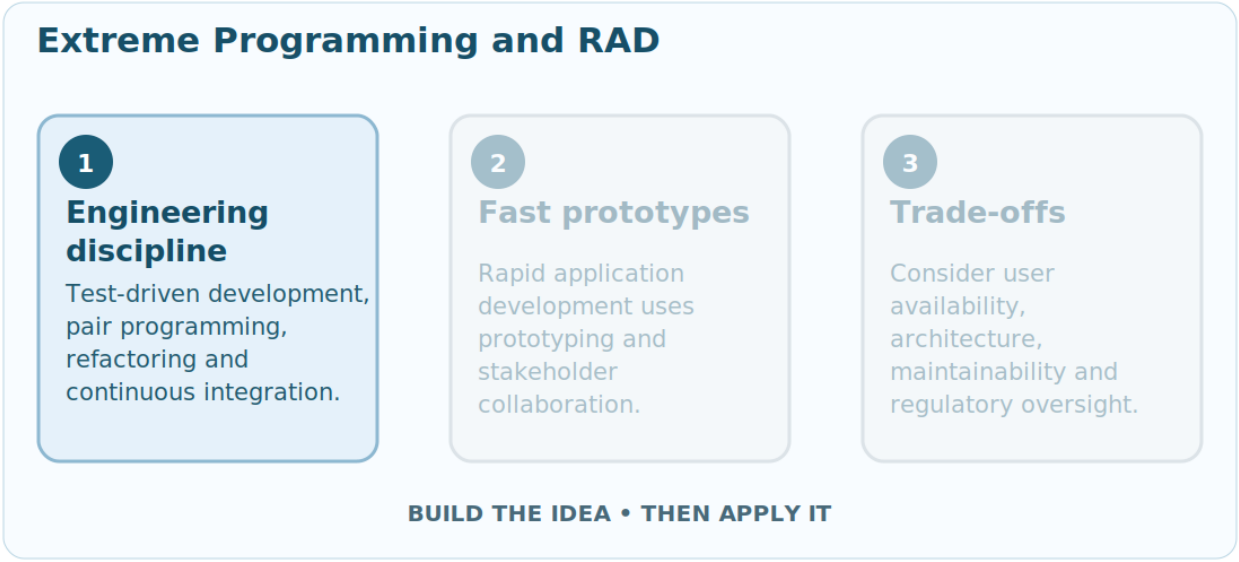
Consider user availability, architecture, maintainability and regulatory oversight.

**Explain one step in your own words, using the correct technical terms.**

# New Learning · original lesson diagrams

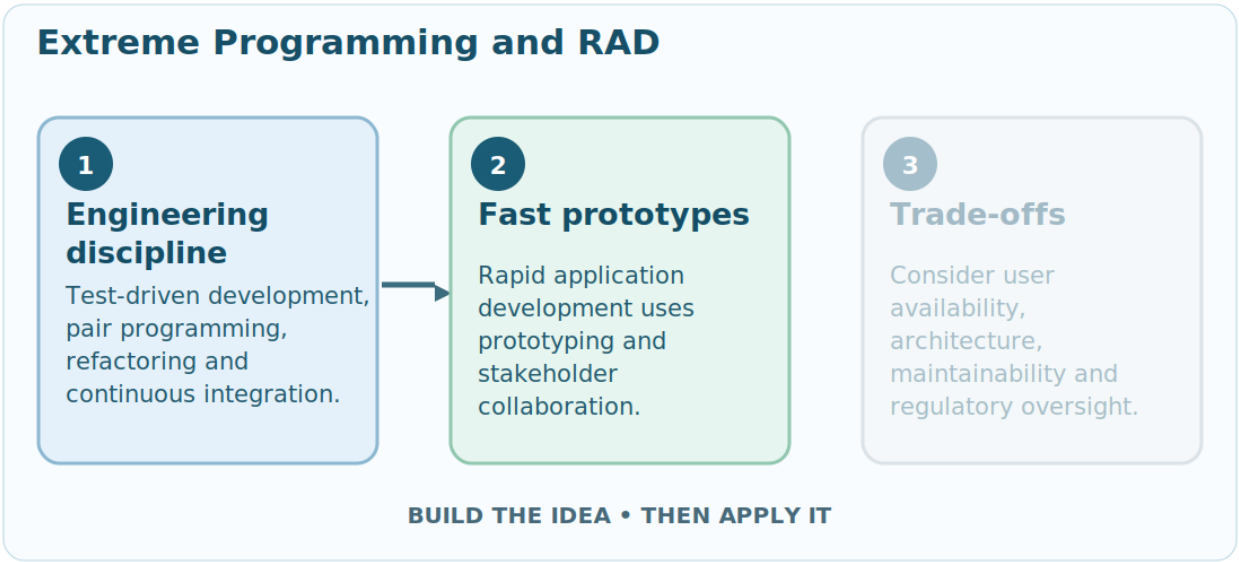
Diagrams appear after the related explanation. Annotate the original source visual.

### New Learning




Extreme Programming and RAD: Engineering discipline then Fast prototypes then Trade-offs

### New Learning

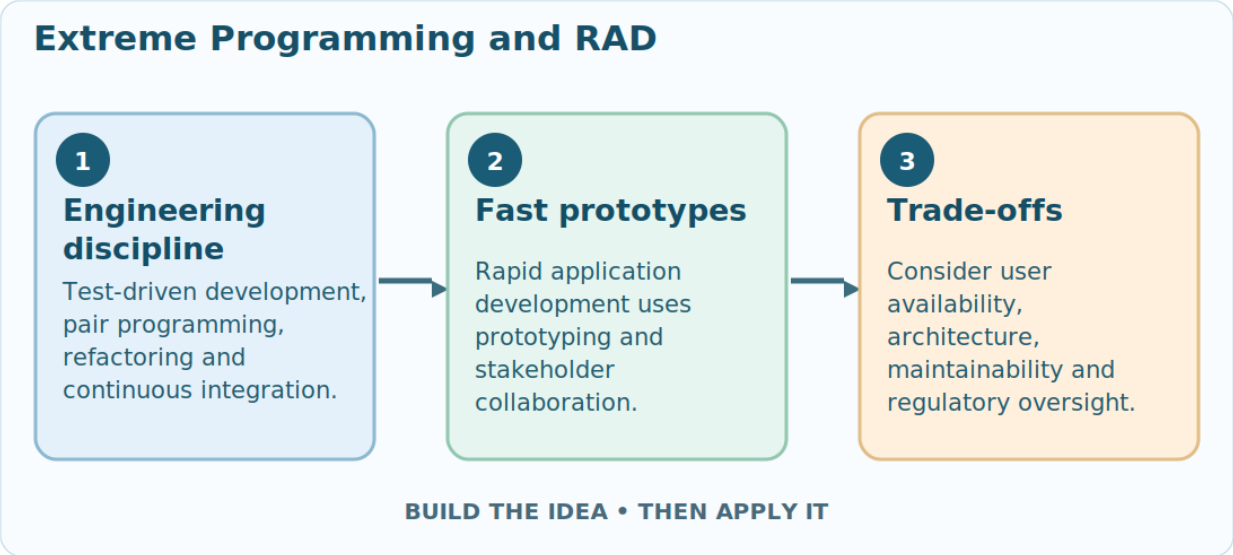



Extreme Programming and RAD: Engineering discipline then Fast prototypes then Trade-offs

# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

### New Learning




Extreme Programming and RAD: Engineering discipline then Fast prototypes then Trade-offs

I DO — WORKED EXAMPLE

# Follow the worked method

### A small team repeatedly breaks existing features after changes. Which XP practices help?

### Model answer

Automated tests written before/with code (TDD) define intended behaviour, continuous integration runs regression tests on each change, and refactoring improves design without changing observable behaviour.

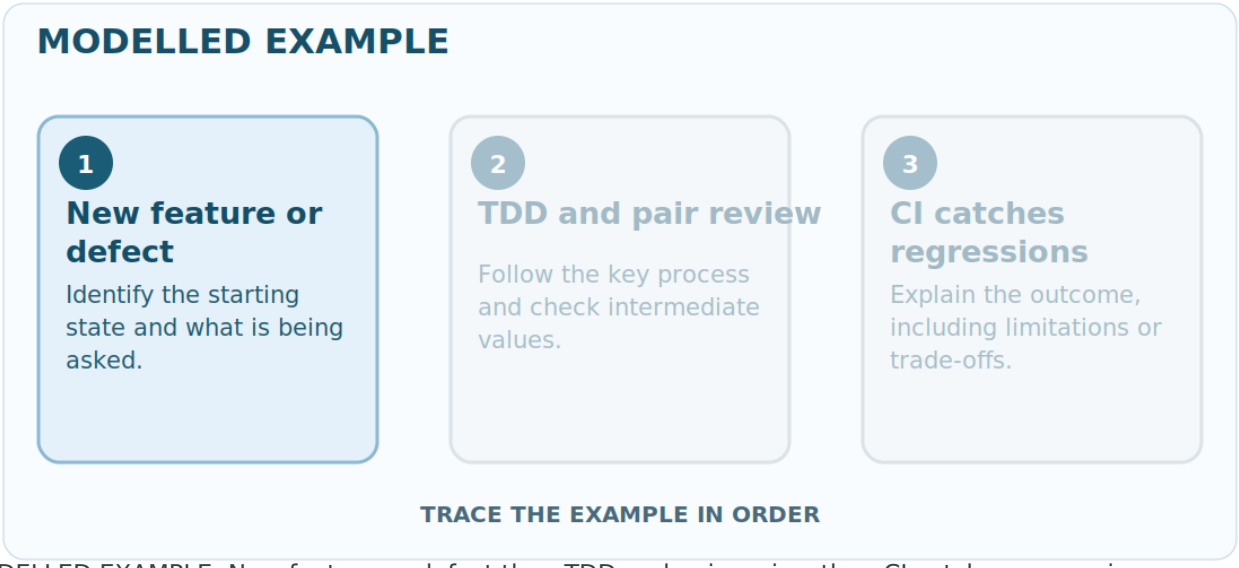
Check: link each claim to the scenario and use precise OCR terminology.

# I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

### A small team repeatedly breaks existing features after changes. Which XP practices help?

Automated tests written before/with code (TDD) define intended behaviour, continuous integration runs regression tests on each change, and refactoring improves design without changing observable behav



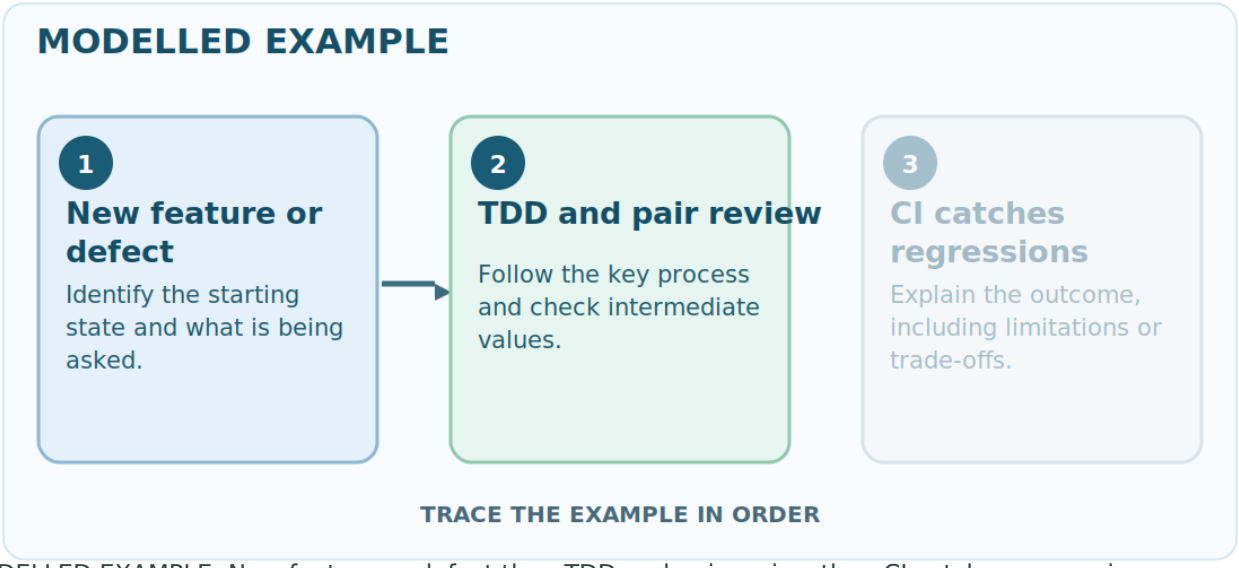

MODELLED EXAMPLE: New feature or defect then TDD and pair review then CI catches regressions

# I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

### A small team repeatedly breaks existing features after changes. Which XP practices help?

Automated tests written before/with code (TDD) define intended behaviour, continuous integration runs regression tests on each change, and refactoring improves design without changing observable behav



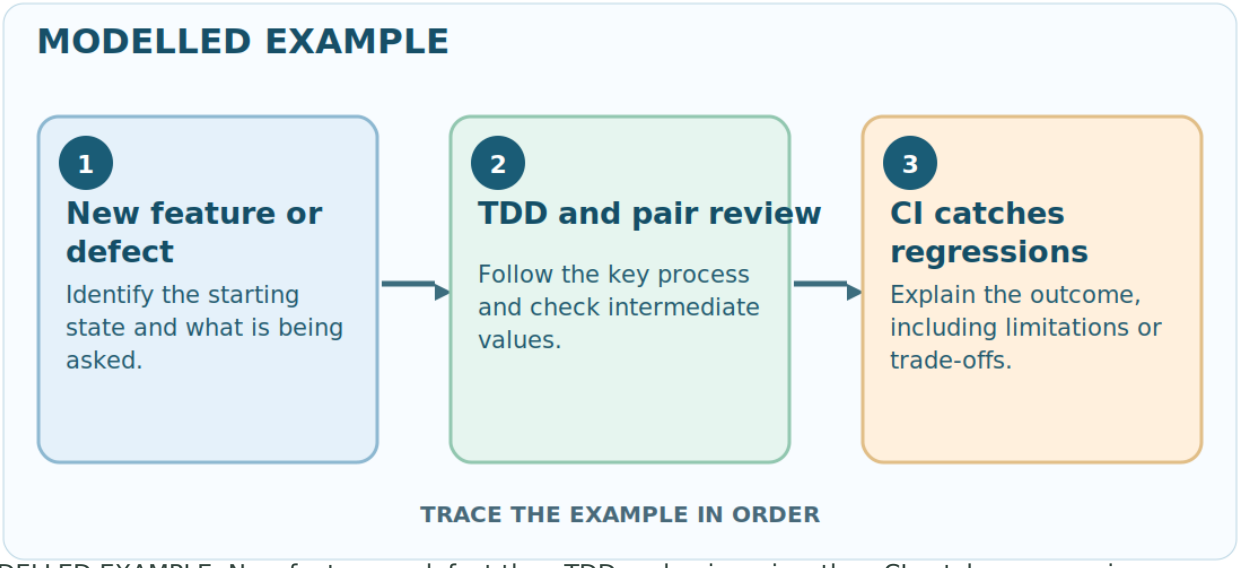

MODELLED EXAMPLE: New feature or defect then TDD and pair review then CI catches regressions

# I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

### A small team repeatedly breaks existing features after changes. Which XP practices help?

Automated tests written before/with code (TDD) define intended behaviour, continuous integration runs regression tests on each change, and refactoring improves design without changing observable behav




MODELLED EXAMPLE: New feature or defect then TDD and pair review then CI catches regressions

WE DO — GUIDED PRACTICE

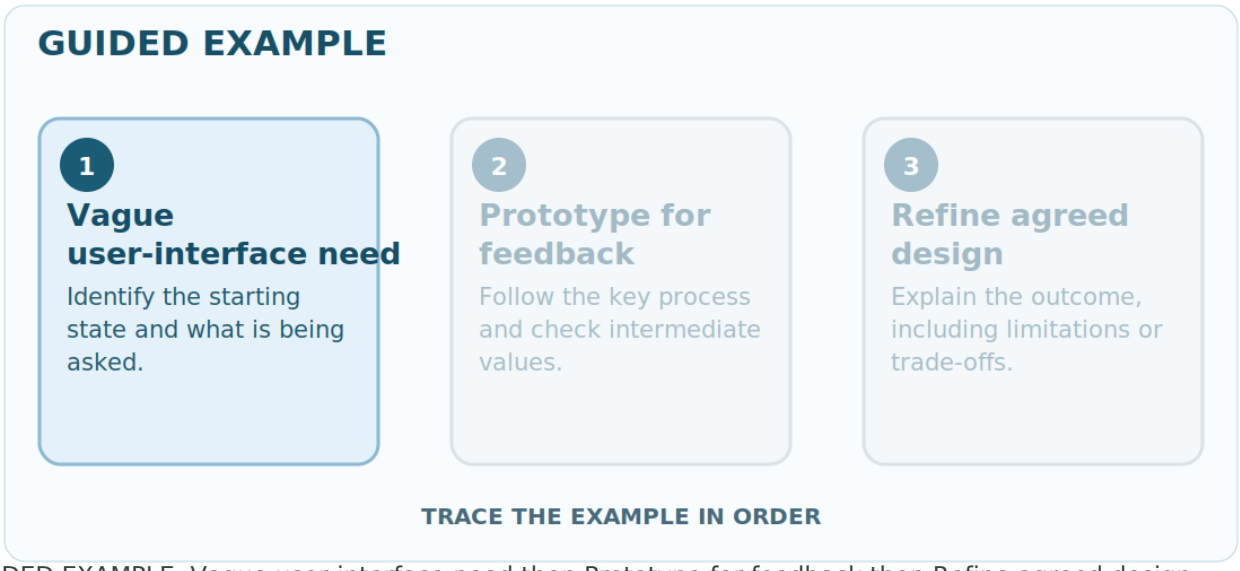
# Try → Discuss → Improve

# We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**A local club cannot describe its booking interface but can test clickable prototypes. Explain RAD.**

Identify the inputs, relevant facts and exact output or judgement required.




GUIDED EXAMPLE: Vague user-interface need then Prototype for feedback then Refine agreed design

# WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

**Guided 1: What must we decide? Identify the inputs, relevant facts and exact output or judgement required.**

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 1

Explain test-driven development and pair programming within XP. [6]

### Question 2

Describe two reasons RAD may suit a system with unclear interface requirements. [6]

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 3

Evaluate XP and RAD for building a safety-critical medication scheduler under time pressure. [8]

EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

**Refactoring is intentionally changing the program's externally visible requirements.**

**How confident are you? Circle one: Confident / Getting there / Need help**

Explain your choice, identifying one concept you can now use independently.

## GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

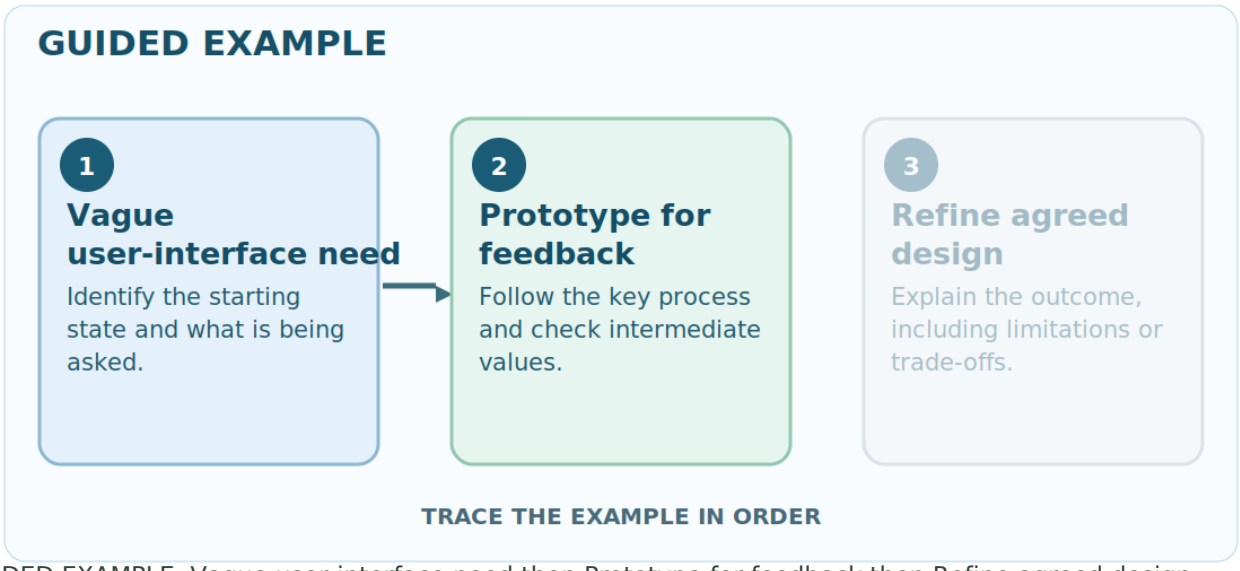
Next step: revisit the model or a relevant revision-guide section if needed.

# We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**A local club cannot describe its booking interface but can test clickable prototypes. Explain RAD.**

Identify the inputs, relevant facts and exact output or judgement required.



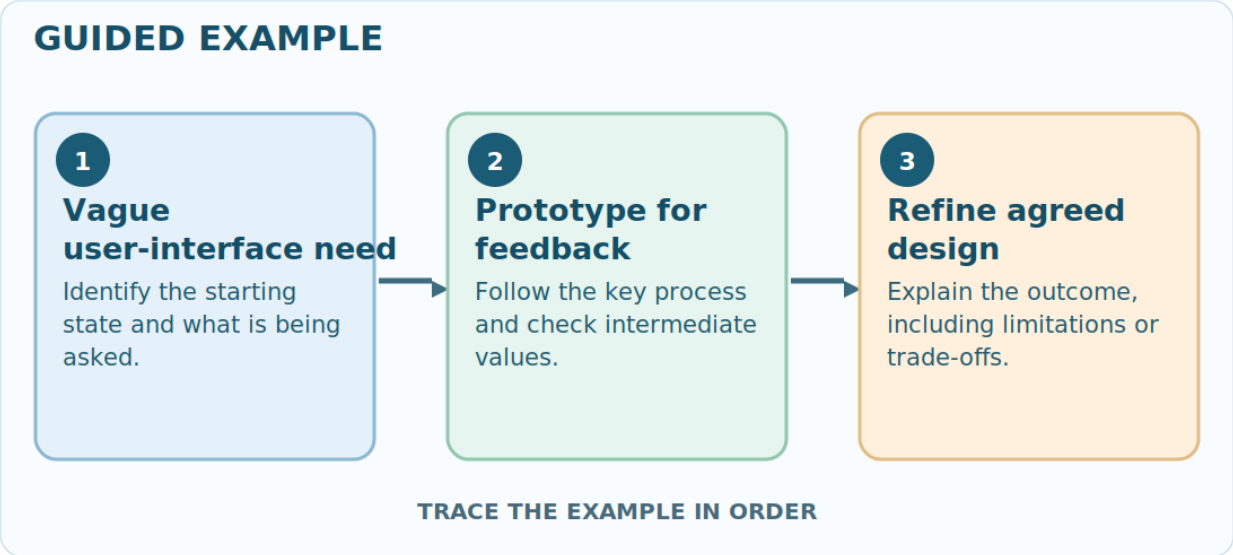

GUIDED EXAMPLE: Vague user-interface need then Prototype for feedback then Refine agreed design

# We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**A local club cannot describe its booking interface but can test clickable prototypes. Explain RAD.**

Identify the inputs, relevant facts and exact output or judgement required.




GUIDED EXAMPLE: Vague user-interface need then Prototype for feedback then Refine agreed design

# WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

### Guided 1 — original CHECK

Work through the method

Use the middle stage of the visual to follow the process and state each intermediate result.

Shared worked answer

Create an early working prototype, invite feedback, revise the interface rapidly and iteratively converge on requirements. Beware treating a rushed prototype as production-quality architecture without further work.

IMPROVE: Explain one correction to your own reasoning.